JEV Flight Lab

External brief ? game, agent harness, Jev I/O, execution, and limitations

Audience: external technical / product partners | Scope: prototype control-loop demo | Decision model: Jev (OpenRouter Decisions API)

1. What the product is

JEV Flight Lab is a Flappy Bird-style 2D browser demo. A neon drone flies through gate openings. The game engine owns physics, rendering, collisions, and scoring. Jev is not the game ? it is a tactical decision service that only answers FLAP or WAIT.

> *Game state -> bounded AI decision -> action -> observable outcome*

The demo shows a real-time control loop with a typed decision model, not chat-based "play the game with prose."

2. Architecture ? who owns what

Layer	Owns	Does not own
Game engine	Physics, gates, collisions, score, lifecycle	AI reasoning
Harness (DecisionManager)	Cadence, timeout, stale checks, fallback, apply/discard	Model weights
Jev provider	Projects state, calls API, parses choice	Execution in the world
Optional LLM analyst	Slow strategic commentary (~10s)	Per-tick control

Critical split: Jev decides; the harness decides whether that answer is still safe to apply; the engine applies a single physics impulse if allowed.

3. End-to-end control loop

```
Animation frame (~60fps)
  -> engine.update(physics)
  -> build TacticalState from live pose
  -> DecisionManager.sync(session, state)
  -> maybeRequest() if interval elapsed & no in-flight call
       -> project state by ~latency lead
       -> POST /api/jev/decision  { state }
       -> server builds Decisions API payload
       -> OpenRouter Jev returns choice flap|wait
  -> harness validates age / relevance / session
  -> if flap + autopilot + still valid -> engine.dispatch(FLAP)
       -> velocityY = flapImpulse (one bounded hop)
  -> if wait -> do nothing (gravity continues)
```

Cadence is adaptive (~40-100 ms target poll), but only one request is in flight, so real rate is bounded by network RTT. Client timeout: 1 s. Max decision age: 1 s.

4. State sent to Jev

Browser -> Next.js route -> OpenRouter. Telemetry is a projected snapshot (lead ~ recent latency, ~200-1000 ms) so Jev answers for near-future pose, not already-stale past.

Numeric telemetry

- altitude, verticalVelocity, horizontalVelocity
- nextGateDistance, gapCenterY, gapSize, relativeGapOffset
- recentFlaps, timeSinceLastFlapMs, droneAlive
- timeToGateMs, predictedOffsetAtGate, gapMargin

Structured prompt fields

- task ? fly through openings; trust ifYouWait; do not invent extra prediction
- ifYouWait ? natural-language forecast if WAIT continues
- oneFlap ? natural-language climb of one flap vs opening height
- questions.action ? choice with explicit flap/wait criteria

Jev is asked for a typed choice, not an explanation.

5. How Jev processes and returns output

1. Server builds OpenRouter Decisions API body (default model typesafe/jev-1.13).

2. Jev returns answers.action.choice = "flap" | "wait", optional probabilities / confidence / usage.
3. Server maps that to TacticalDecision and returns JSON to the browser.
4. Invalid choice / HTTP failure -> harness backup (local model or deterministic heuristic), labeled fallback/local, never as live Jev.

6. How the agent executes the output

Answer	Execution
wait	No control action; gravity keeps acting
flap	If still valid under harness rules -> FLAP -> sets velocityY = flapImpulse once

The harness may discard even a "correct" Jev flap when:

- decision age > 1 s (too_old)
- drone already above the opening enough that a flap would climb into the pipe (above_opening)
- a second impulse too soon after another flap (second_flap, ~280 ms quiet window)
- response belongs to a previous flight/session (previous_flight / replaced)

Model output != world action. The harness is the safety / relevance gate.

7. Key limitations and challenges

- Latency vs game speed. Gates close in seconds; Jev RTT is often hundreds of ms. Mitigation: state projection + stale discard ? projection is approximate, not a full simulator inside Jev.
- Binary, myopic action space. Only flap/wait for the current opening. No multi-gate planning, thrust magnitude, or continuous control.
- Prompted heuristics in the loop. ifYouWait / oneFlap / criteria are app-authored summaries. Results partly reflect prompt engineering, not raw pixel perception.
- No visual perception. Jev never sees the canvas ? only derived numbers + natural language. Feature extraction errors become model errors.
- Fallback opacity vs truthfulness. Timeouts use heuristic/local backup so the drone keeps flying. UI must label fallback, or demos overclaim "Jev flew."
- Non-determinism and eval hardness. Same state can yield different choices; live score is a noisy KPI. Replay/benchmark tooling helps.
- Stale-discard tradeoff. Aggressive discard avoids late ceiling flaps, but throws away expensive answers and can under-control under high latency.
- Scope boundary. This is a lab demo of structured real-time decisions ? not production autonomy or sim-to-real transfer.

8. Stakeholder summary

> *We run a tiny real-time flight sim; every few dozen milliseconds a harness asks Jev "flap or wait?" with projected telemetry and fixed criteria; Jev returns a typed choice; the harness only applies flaps that are still relevant to the live pose, otherwise falls back or waits ? making the demo about bounded, inspectable decision latency, not open-ended LLM gameplay.*

Generated from the JEV Flight Lab implementation (DecisionManager, jevServer, game engine control loop). For implementation detail, see PROJECT_SPEC.md and TECHNICAL_IMPLEMENTATION.md.